

SIH 2026

DEVELOPER 1 — DATA & STORAGE MICROSERVICE PRD

PostgreSQL • Redis • IPFS • Persistence • Read Models

Architecture context: *This PRD defines one independently owned microservice in the agreed three-service architecture. The services are Data & Storage, Blockchain, and Application/Backend. They communicate through explicit internal contracts.*

40-hour Base Model: Build the smallest complete vertical slice first. EPCIS/event streaming, IoT, AI/ML and advanced privacy remain future modules.

1. Mission

The Data & Storage Service owns the persistence boundary: PostgreSQL, Redis and IPFS. It exposes stable internal APIs to the Application Service. It stores structured operational/read-model data, lineage, cached reads and evidence metadata/objects. It does not own business workflows or Hyperledger Fabric.

2. Ownership

Owned by Developer 1	Explicitly not owned
PostgreSQL schema, migrations, repositories	FastAPI business logic
Redis and cache abstraction	Fabric network/chaincode
IPFS and evidence integration	Frontend
Operational/read-model persistence	Application-level risk decisions
Data Service internal API	Future streaming/IoT/AI modules

3. Architecture

Application Service → internal REST API → Data Service → {PostgreSQL, Redis, IPFS}

4. PostgreSQL requirements

1. Persist organizations, users, roles and permissions data required by the application.
2. Persist products, batches and units.
3. Persist business/custody events and scan/verification read models.
4. Persist parent-child lineage relationships for bidirectional traversal.
5. Persist QR credential/reference metadata.
6. Persist incidents, feedback evidence metadata, accountability and escalations.
7. Persist risk scopes and recall-action read models.
8. Persist Fabric transaction IDs/event references for correlation.
9. Use migrations, constraints and indexes; other services must not alter the schema ad hoc.

5. Logical schema

Domain	Core entities
Identity	organizations, users, roles, permissions
Traceability	products, batches, units
History	events, custody_events, scan_events
Lineage	lineage_edges
QR	qr_credentials
Feedback	incidents, feedback_evidence, evidence
Accountability	accountability_records, escalations
Risk/Recall	risk_scopes, risk_scope_nodes, recall_actions
Audit	audit_log

6. Lineage

Lineage is a graph relationship. The service must support upstream and downstream traversal and expose a stable internal API. Parent and child references must be indexed and traversal must be bounded.

Entity → lineage_edges → parent(s) / child(ren) → recursive/iterative traversal → trace/risk result

7. Redis

1. Redis is a rebuildable cache, never the source of truth.
2. Provide versioned cache keys.
3. Cache trace/history, batch/unit, QR resolution and risk-scope reads where useful.

4. Invalidate affected keys after trusted mutation has been synchronized to PostgreSQL.
5. Fall back to PostgreSQL on cache miss or Redis failure.

trace:{entity}:{id}:v1 | batch:{id}:v1 | unit:{id}:v1 | qr:{ref}:v1 | risk:{id}:v1

8. IPFS

1. Accept evidence objects through the internal API.
2. Return/store CID and searchable metadata.
3. Support certificates, lab reports, feedback attachments and incident evidence.
4. Do not put large files into PostgreSQL or Fabric.
5. Retrieve evidence only through the service authorization boundary.

9. Internal API

Method	Endpoint	Purpose
POST	/internal/products	Create/update product read model
GET	/internal/products/{id}	Read product
POST	/internal/batches	Create/update batch read model
GET	/internal/batches/{id}	Read batch
GET	/internal/lineage/{id}	Read lineage
POST	/internal/events	Persist trusted event read model
POST	/internal/scans	Persist scan read model
POST	/internal/incidents	Persist incident read model
POST	/internal/evidence	Upload evidence / CID metadata
GET	/internal/evidence/{cid}	Retrieve evidence
POST	/internal/risk-scopes	Persist risk scope
POST	/internal/recalls	Persist recall read model

10. Consistency

Application → Blockchain Service → Fabric COMMIT → Application → Data Service → PostgreSQL → Redis invalidation

Data synchronization should be idempotent where possible. A repeated commit synchronization with the same transaction ID must not create duplicate read-model records.

11. Security

1. Do not expose PostgreSQL, Redis or IPFS administration publicly.
2. Authenticate internal service requests.
3. Do not store Fabric private keys in this service.
4. Do not store secrets in application data.
5. Authorize restricted evidence retrieval.
6. Return only consumer-safe fields when requested by the Application Service.

12. Phase 1 deliverables

1. Runnable Data Service.
2. PostgreSQL migration baseline and core repositories.
3. Redis adapter/cache abstraction.
4. IPFS adapter/evidence API.
5. Seed/demo data.
6. Lineage queries.
7. Internal OpenAPI contract.
8. Health checks and integration tests.

13. Definition of Done

Another developer can run the Data Service independently, create/read Product and Batch data, traverse lineage, upload/retrieve IPFS evidence, use Redis as a rebuildable cache, and consume the documented internal API without direct access to its infrastructure.